

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in dark ink. The signature reads "D. Gnanadeepthi." in a cursive, slightly slanted script.

QUESTIONS Marks: 50			Total
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5

9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

ANSWERS

1. Employee(EmpID, EmpName, Salary, DeptID)

EmpID	EmpName	Salary	DeptID
E1	Arun	50000	D1
E2	Priya	60000	D2
E3	Rahul	45000	D1
E4	Sneha	70000	D3

Department(DeptID, DeptName)

DeptID	DeptName
D1	HR
D2	IT
D3	Finance

1. Selection (σ)

Retrieve employees with salary greater than 50,000.

Relational Algebra:

$$\sigma_{Salary > 50000}(Employee)$$

EmpID	EmpName	Salary	DeptID
-------	---------	--------	--------

E2	Priya	60000	D2
----	-------	-------	----

E4	Sneha	70000	D3
----	-------	-------	----

2. Projection (π)

Retrieve only employee names.

Relational Algebra:

$$\pi_{EmpName}(Employee)$$

EmpName

Arun

Priya

Rahul

Sneha

3. Union ($\cup$)

Assume another relation:

Employee_New(EmpID, EmpName, Salary, DeptID)

EmpID	EmpName	Salary	DeptID
-------	---------	--------	--------

E5	Kiran	55000	D2
----	-------	-------	----

E6	Neha	65000	D3
----	------	-------	----

Relational Algebra:

$$Employee \cup Employee_New$$

Result:

EmpID EmpName Salary DeptID

E1	Arun	50000	D1
E2	Priya	60000	D2
E3	Rahul	45000	D1
E4	Sneha	70000	D3
E5	Kiran	55000	D2
E6	Neha	65000	D3

4. Set Difference (-)

Find employees in Employee but not in Employee_New.

Relational Algebra:

Employee – Employee_New

EmpID EmpName Salary DeptID

E1	Arun	50000	D1
E2	Priya	60000	D2
E3	Rahul	45000	D1
E4	Sneha	70000	D3

5. Cartesian Product (×)

Combine every employee with every department.

Relational Algebra:

Employee × Department

Number of tuples:

- Employee = 4 rows
- Department = 3 rows
- Result = $4 \times 3 = 12$ tuples

Sample Result:

EmpName DeptID(Employee) DeptID(Department) DeptName

Arun	D1	D1	HR
Arun	D1	D2	IT
Arun	D1	D3	Finance
Priya	D2	D1	HR
...	...	...	...

6. Join (⋈)

Retrieve employee names along with their department names.

Relational Algebra:

Employee ⋈_{Employee.DeptID=Department.DeptID} Department

Result:

EmpID EmpName Salary DeptName

E1	Arun	50000	HR
E2	Priya	60000	IT
E3	Rahul	45000	HR
E4	Sneha	70000	Finance

2.

CREATE TABLE DEPARTMENT (

DeptID INT PRIMARY KEY,

DeptName VARCHAR(50) NOT NULL UNIQUE

);

CREATE TABLE STUDENT (

SID INT PRIMARY KEY,

Name VARCHAR(50) NOT NULL,
DOB DATE NOT NULL,
DeptID INT,
CONSTRAINT FK_Dept
FOREIGN KEY (DeptID)
REFERENCES DEPARTMENT(DeptID)

);

CONSTRAINTS USED:

- **PRIMARY KEY:** Ensures each record is unique (SID, DeptID).
- **NOT NULL:** Prevents Name, DOB, and DeptName from being empty.
- **UNIQUE:** Ensures no duplicate department names.
- **FOREIGN KEY:** Links STUDENT.DeptID to DEPARTMENT.DeptID, maintaining referential integrity.

SAMPLE OUTPUT:

Table DEPARTMENT created.

Table STUDENT created.

3. Assume the following Sales table:

SALES(SaleID, Product, Category, Quantity, Amount)

SaleID	Product	Category	Quantity	Amount
101	Laptop	Electronics	2	100000
102	Mouse	Electronics	5	5000
103	Chair	Furniture	3	15000
104	Table	Furniture	2	20000
105	Mobile	Electronics	4	80000

SaleID	Product	Category	Quantity	Amount
106	Sofa	Furniture	1	25000

1. COUNT()

Count the number of products in each category.

```
SELECT Category, COUNT(*) AS Total_Products
FROM SALES
GROUP BY Category;
```

Category	Total_Products
Electronics	3
Furniture	3

2. SUM()

Find the total sales amount for each category.

```
SELECT Category, SUM(Amount) AS Total_Sales
FROM SALES
GROUP BY Category;
```

Category	Total_Sales
Electronics	185000
Furniture	60000

3. AVG()

Find the average sales amount in each category.

```
SELECT Category, AVG(Amount) AS Average_Sales
FROM SALES
GROUP BY Category;
```


Category Average_Sales

Electronics 61666.67

Furniture 20000.00

4. MAX()

Find the maximum sales amount in each category.

```
SELECT Category, MAX(Amount) AS Maximum_Sales
FROM SALES
GROUP BY Category;
```

Category Maximum_Sales

Electronics 100000

Furniture 25000

5. MIN

Find the minimum sales amount in each category.

```
SELECT Category, MIN(Amount) AS Minimum_Sales
FROM SALES
GROUP BY Category;
```

Category Minimum_Sales

Electronics 5000

Furniture 15000

6. GROUP BY with HAVING

Display categories whose total sales amount is greater than 1,00,000.

```
SELECT Category, SUM(Amount) AS Total_Sales
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 100000;
```

Category Total_Sales

Electronics 185000

4. Correlated Subquery

Objective: Find employees whose salary is greater than the average salary of their own department.

Assume the EMPLOYEE table has the following columns:

```
SELECT EmpID, EmpName, Salary, DeptID
```

```
FROM EMPLOYEE E1
```

```
WHERE Salary > (
```

```
    SELECT AVG(Salary)
```

```
    FROM EMPLOYEE E2
```

```
    WHERE E1.DeptID = E2.DeptID
```

```
);
```

Explanation

- E1 represents the outer query.
- E2 represents the inner (correlated) query.
- The inner query calculates the average salary for the department of the current employee.
- The outer query returns only those employees whose salary is greater than the average salary of their own department.

Example

EMPLOYEE

EmpID	EmpName	Salary	DeptID
101	Arun	50000	D1
102	Priya	60000	D1
103	Rahul	45000	D2
104	Sneha	70000	D2

EmpID	EmpName	Salary	DeptID
105	Kiran	55000	D1

RESULT:

EmpID	EmpName	Salary	DeptID
102	Priya	60000	D1
104	Sneha	70000	D2

5.

EMPLOYEE

EmpID	EmpName	ProjectID
101	Arun	P1
102	Priya	P2
103	Rahul	P3
104	Sneha	NULL

PROJECT:

ProjectID	ProjectName
P1	Library Management
P2	Banking System
P4	Hospital Management

1. INNER JOIN

Returns only the matching records from both tables.

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
```

INNER JOIN PROJECT P

ON E.ProjectID = P.ProjectID;

OUTPUT:

EmpID	EmpName	ProjectName
101	Arun	Library Management
102	Priya	Banking System

2. LEFT OUTER JOIN

Returns all employees and matching project details. If no matching project exists, NULL is displayed.

SELECT E.EmpID, E.EmpName, P.ProjectName

FROM EMPLOYEE E

LEFT OUTER JOIN PROJECT P

ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectName
101	Arun	Library Management
102	Priya	Banking System
103	Rahul	NULL
104	Sneha	NULL

3. RIGHT OUTER JOIN

Returns all projects and matching employee details. If no employee is assigned, employee columns are NULL.

SELECT E.EmpID, E.EmpName, P.ProjectName

FROM EMPLOYEE E

RIGHT OUTER JOIN PROJECT P

ON E.ProjectID = P.ProjectID;

EmpID	EmpName	ProjectName
101	Arun	Library Management
102	Priya	Banking System
NULL	NULL	Hospital Management

4. FULL OUTER JOIN

Returns all records from both tables. Matching rows are combined, and unmatched rows contain NULL.

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

OUTPUT:

EmpID	EmpName	ProjectName
101	Arun	Library Management
102	Priya	Banking System
103	Rahul	NULL
104	Sneha	NULL
NULL	NULL	Hospital Management

5. CROSS JOIN

Returns the Cartesian product of both tables (every employee with every project).

OUTPUT:

EmpName	ProjectName
Arun	Library Management
Arun	Banking System
Arun	Hospital Management

EmpName	ProjectName
Priya	Library Management
Priya	Banking System
Priya	Hospital Management
Rahul	Library Management
Rahul	Banking System
Rahul	Hospital Management
Sneha	Library Management
Sneha	Banking System
Sneha	Hospital Management

6.

- **EMPLOYEE(EmpID, EmpName, Salary, DeptID)**
- **DEPARTMENT(DeptID, DeptName)**

1. Create the View DeptSalaryView

CREATE VIEW DeptSalaryView AS

SELECT

D.DeptName,

COUNT(E.EmpID) AS Total_Employees,

AVG(E.Salary) AS Average_Salary

FROM DEPARTMENT D

JOIN EMPLOYEE E

ON D.DeptID = E.DeptID

GROUP BY D.DeptName;

2. Query on the View

Display all department salary details

```
SELECT * FROM DeptSalaryView;
```

DeptName	Total_Employees	Average_Salary
HR	5	45000.00
IT	8	62000.00
Finance	4	55000.00

Example Query: Display departments with average salary greater than 50,000

```
SELECT *
```

```
FROM DeptSalaryView
```

```
WHERE Average_Salary > 50000;
```

DeptName	Total_Employees	Average_Salary
IT	8	62000.00
Finance	4	55000.00

7.

Assume the following relations:

- **STUDENT(SID, Name)**
- **COURSE(CID, CourseName)**
- **ENROLLMENT(SID, CID)**

Relational Algebra Expression

To find the **names of students who are enrolled in all courses offered**, use the **Division ($\div$)** operator.

$$\pi_{Name} \left(STUDENT \bowtie \left(\pi_{SID,CID}(ENROLLMENT) \div \pi_{CID}(COURSE) \right) \right)$$

Step-by-Step Explanation

1. **Get all course IDs:**

$$\pi_{CID}(COURSE)$$

2. **Get student-course pairs:**

$$\pi_{SID,CID}(ENROLLMENT)$$

3. **Apply Division ($\div$):**

$$\pi_{SID,CID}(ENROLLMENT) \div \pi_{CID}(COURSE)$$

This returns the **SIDs of students who are enrolled in every course**.

4. **Join with STUDENT and project names:**

$$\pi_{Name} \left(STUDENT \bowtie \left(\pi_{SID,CID}(ENROLLMENT) \div \pi_{CID}(COURSE) \right) \right)$$

Final Answer

$$\boxed{\pi_{Name} \left(STUDENT \bowtie \left(\pi_{SID,CID}(ENROLLMENT) \div \pi_{CID}(COURSE) \right) \right)}$$

Note: The **Division ($\div$)** operator is specifically used in relational algebra to answer queries of the form "**find all entities related to every item in another set**", making it the correct choice for this problem.

8.

Scenario

Assume the following table:

STUDENT(

SID INT PRIMARY KEY,

Name VARCHAR(50) NOT NULL,

DOB DATE,

DeptID INT,

FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)

);

1. INSERT Statement

Insert a new student record.

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES (101, 'Rahul', '2005-06-15', 1);
```

2. UPDATE Statement

Update the department of student with SID = 101.

```
UPDATE STUDENT
SET DeptID = 2
WHERE SID = 101;
```

3. DELETE Statement

Delete the student with SID = 101.

```
DELETE FROM STUDENT
WHERE SID = 101;
```

EXAMPLE:

BEFORE OPERATIONS:

SID	Name	DOB	DeptID
101	Rahul	2005-06-15	1
102	Priya	2004-12-20	2

AFTER UPDATE:

SID	Name	DOB	DeptID
101	Rahul	2005-06-15	2
102	Priya	2004-12-20	2

AFTER DELETE:

SID	Name	DOB	DeptID
102	Priya	2004-12-20	2

9.

Assume the following relations:

- **EMPLOYEE**(EmpID, EmpName, Salary, DeptID)
- **DEPARTMENT**(DeptID, DeptName)

Tuple Relational Calculus (TRC) Expression

$$\{ e. EmpName \mid EMPLOYEE(e) \wedge \exists d (DEPARTMENT(d) \wedge e. DeptID = d. DeptID \wedge d. DeptName = 'IT' \wedge e. Salary > 50000) \}$$

Explanation

- **EMPLOYEE(e)** → e is a tuple from the **EMPLOYEE** relation.
- **DEPARTMENT(d)** → d is a tuple from the **DEPARTMENT** relation.
- **e.DeptID = d.DeptID** → Matches each employee with their department.
- **d.DeptName = 'IT'** → Selects employees who work in the IT department.
- **e.Salary > 50000** → Filters employees earning more than 50,000.
- **e.EmpName** → Returns only the employee names.

Final Answer

$$\boxed{\{ e. EmpName \mid EMPLOYEE(e) \wedge \exists d (DEPARTMENT(d) \wedge e. DeptID = d. DeptID \wedge d. DeptName = 'IT' \wedge e. Salary > 50000) \}}$$

10.

Airline Reservation System – Relational Schema

1. Relational Schema

AIRLINE

AIRLINE(AirlineID, AirlineName)

- AirlineID (Primary Key)

FLIGHT

FLIGHT(FlightID, FlightName, Source, Destination, AirlineID)

- FlightID (Primary Key)
- AirlineID (Foreign Key → AIRLINE.AirlineID)

PASSENGER

PASSENGER(PassengerID, PassengerName, Age, Gender, Phone)

- PassengerID (Primary Key)

RESERVATION

RESERVATION(ReservationID, PassengerID, FlightID, JourneyDate, SeatNo, TicketFare)

- ReservationID (Primary Key)
- PassengerID (Foreign Key → PASSENGER.PassengerID)
- FlightID (Foreign Key → FLIGHT.FlightID)

SQL Queries

Query 1 – INNER JOIN

Display passenger names along with their flight details.

```
SELECT P.PassengerName,  
       F.FlightName,  
       F.Source,  
       F.Destination  
FROM PASSENGER P  
INNER JOIN RESERVATION R  
ON P.PassengerID = R.PassengerID  
INNER JOIN FLIGHT F  
ON R.FlightID = F.FlightID;
```

Query 2 – JOIN with Aggregate Function

Display the number of passengers traveling on each flight.

```
SELECT F.FlightName,  
       COUNT(R.PassengerID) AS TotalPassengers  
FROM FLIGHT F  
LEFT JOIN RESERVATION R  
ON F.FlightID = R.FlightID  
GROUP BY F.FlightName;
```

Query 3 – Subquery

Find passengers who paid more than the average ticket fare.

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN (
    SELECT PassengerID
    FROM RESERVATION
    WHERE TicketFare >
        (SELECT AVG(TicketFare)
         FROM RESERVATION)
);
```

Query 4 – Correlated Subquery

Find flights having more reservations than the average number of reservations per flight.

```
SELECT FlightName
FROM FLIGHT F
WHERE (
    SELECT COUNT(*)
    FROM RESERVATION R
    WHERE R.FlightID = F.FlightID
) >
(
    SELECT AVG(FlightCount)
    FROM (
        SELECT COUNT(*) AS FlightCount
        FROM RESERVATION
        GROUP BY FlightID
    ) AS T
);
```

Query 5 – View

Create a view showing flight details and total reservations.

```
CREATE VIEW FlightReservationView AS  
SELECT F.FlightID,  
       F.FlightName,  
       COUNT(R.ReservationID) AS TotalReservations  
FROM FLIGHT F  
LEFT JOIN RESERVATION R  
ON F.FlightID = R.FlightID  
GROUP BY F.FlightID, F.FlightName;  
  
SELECT *  
FROM FlightReservationView;
```

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding ; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation